

Memory-Efficient Torch Inference for AlphaGenome

July 3, 2026

Abstract

We evaluated low-VRAM Torch inference strategies for AlphaGenome on a recreated human brain development ATAC checkpoint. At 131 kb and batch size 32, the lowest-memory configuration reduced observed peak memory from 90.46 GiB to 24.59 GiB, with test r changing from 0.80891 to 0.80860. At 1 Mb and batch size 2, default inference exceeded 48 GiB, whereas the complete fused configuration used 23.58 GiB.

1 Experimental setup

All experiments used the recreated merged Torch checkpoint from the best LoRA+LoCon finetuning run. We evaluated the human brain development ATAC head at resolution 128 on full valid and test splits. For 131 kb input windows, batch size was 32 and the evaluation contained 2162 examples. For 1 Mb input windows, a valid/test-only target cache was built, batch size was 2, and the evaluation contained 269 examples. Peak device memory was sampled with `nvidia-smi` and is reported as GiB. Accuracy is reported as test differential Pearson. Throughput is reported as examples per second over the full valid/test evaluation.

2 Implementation details

Bfloat16 policy. The `bf16_params` path uses the Torch model’s aggressive bfloat16 dtype policy for inference. Floating-point model state and persistent activations are stored or cast to bfloat16 where supported by the Torch implementation, while integer inputs and indexing tensors are unchanged.

Effective standardized convolutions. AlphaGenome standardizes convolution weights at every forward pass. For an output channel, the effective inference weight is

$$W_{\text{eff}} = (W - \text{mean}(W)) \text{rsqrt}(\max(\text{var}(W) \text{fan_in}, 10^{-4})) s,$$

where s is the learned scale. The `_stdconv_effective` path materializes W_{eff} once and replaces the module with a direct convolution that preserves AlphaGenome SAME padding. This avoids runtime weight standardization and makes the layer eligible for the Triton Conv1d path.

Triton int8 weight-only Conv1d. Eligible wide Conv1d layers are replaced by a custom Triton kernel. Weights are stored as symmetric int8 values with one scale per output channel:

$$\alpha_c = \frac{\max |W_c|}{127}, \quad Q_c = \text{round}(W_c / \alpha_c).$$

The kernel dequantizes inside the matrix multiply, accumulates in fp32 using `tl.dot`, and stores outputs in the input activation dtype. Heads, adapters, LoRA/LoCon modules, and other sensitive paths are excluded.

No-intermediates encoder path. When only 128 bp predictions are requested, the decoder does not consume the encoder U-Net skip tensors. The patched encoder therefore does not retain these intermediate tensors during evaluation.

Triton no-indices max-pooling. The encoder pool has kernel size 2 and stride 2. The custom Triton kernel loads the two candidate positions and stores only their maximum, matching the local ceil-length SAME pooling behavior while avoiding PyTorch’s hidden max-pool index/workspace allocation.

Fused DNA embedder block. The `encoder.dna_embedder.block` module is fused into one Triton kernel implementing

$$\text{RMSBatchNorm} \rightarrow \text{GELU}_{\text{JAX}} \rightarrow \text{Conv1d}_{\text{int8 weight-only}}.$$

The RMSBatchNorm step uses stored running variance and affine parameters,

$$x_{\text{norm}} = x \cdot \gamma \cdot \text{rsqrt}(\text{running_var} + \epsilon) + \beta.$$

The activation is AlphaGenome/JAX’s sigmoid approximation rather than the standard erf GELU:

$$\text{GELU}_{\text{JAX}}(x) = x \cdot \sigma(1.702x).$$

The fused kernel applies normalization and activation immediately before the int8 weight-only convolution, without materializing the full-resolution normalized or activated tensors.

This saves VRAM because the unfused path creates separate full-resolution temporary tensors for normalization, activation, and convolution input. At 131 kb and batch size 32, these temporaries are large enough to drive allocator reservation and observed peak memory. The fused kernel streams through the operation and only stores the final block output.

Fused first downsampling block. The `encoder.down_blocks.0` module is replaced by a wrapper containing two fused ConvBlocks and lean residual accumulation. In inference, the first block output already has the expanded channel dimension, so the residual input is added in-place into the leading input-channel slice instead of constructing an explicitly padded residual tensor. The second residual add is also in-place. This saves VRAM by avoiding both the ConvBlock temporaries described above and the explicit padded residual tensor for the first channel-expanding down block. The benefit appears mostly as lower reserved/observed device memory because the peak allocator state is sensitive to high-resolution transient tensors.

3 Results

Tables use feature indicators for bfloat16 policy (BF16), effective standardized convolutions (Eff.), Triton int8 Conv1d (Int8), no-intermediates encoder path (NoInt), Triton no-indices max-pool (Pool), fused DNA embedder block (FEmb), and fused first downsampling block (FD0).

Table 1: Full valid/test evaluation for 131 kb windows at batch size 32.

Strategy	BF16	Eff.	Int8	NoInt	Pool	FEmb	FD0	Peak GiB	Ex./s	Test r
Default								90.46	9.41	0.80891
Bfloat16	✓							65.37	12.02	0.80874
Triton Conv1d	✓	✓	✓					42.59	8.49	0.80877
No intermediates	✓	✓	✓	✓				42.59	8.51	0.80877
Triton pool	✓	✓	✓	✓	✓			34.09	8.78	0.80877
Fused embedder	✓	✓	✓	✓	✓	✓		31.59	8.03	0.80872
Fused down0	✓	✓	✓	✓	✓		✓	30.59	8.11	0.80860
Fused embedder+down0	✓	✓	✓	✓	✓	✓	✓	24.59	7.45	0.80860

Table 2: Full valid/test evaluation for 1 Mb windows at batch size 2.

Strategy	BF16	Eff.	Int8	NoInt	Pool	FEmb	FD0	Peak GiB	Ex./s	Test r
Default								51.36	0.721	0.82707
Bfloat16	✓							46.48	0.937	0.82640
Triton Conv1d	✓	✓	✓					32.58	0.736	0.82662
No intermediates	✓	✓	✓	✓				28.58	0.749	0.82662
Triton pool	✓	✓	✓	✓	✓			30.08	0.753	0.82662
Fused embedder	✓	✓	✓	✓	✓	✓		28.83	0.701	0.82658
Fused down0	✓	✓	✓	✓	✓		✓	26.58	0.697	0.82643
Fused embedder+down0	✓	✓	✓	✓	✓	✓	✓	23.58	0.650	0.82634

At 131 kb, the lowest-memory successful configuration was the fused embedder+down0 strategy, reducing observed peak memory from 90.46 GiB to 24.59 GiB relative to default. The best non-fused memory-efficient option was the Triton-pool strategy, at 34.09 GiB with no visible test metric change relative to the Triton Conv1d baseline.

At 1 Mb, default inference exceeded 48 GiB. Plain bfloat16 inference fit just under 48 GiB, while the Triton/no-intermediates family reduced memory substantially further. The fused blocks primarily reduced observed memory at this longer context length; they are useful for minimum VRAM but not for maximum throughput.

3.1 1 Mb FlexAttention follow-up

FlexAttention was re-tested at 1 Mb because attention and attention-bias tensors become more important at longer context. These runs used the same batch size 2, checkpoint, target cache, and `bf16_triton_conv_stdconv_effective` baseline.

Table 3: 1 Mb attention-backend follow-up on top of the Triton Conv1d baseline.

Attention path	Bias	Peak GiB	Alloc GiB	Reserved GiB	Ex./s	Test r
Eager baseline	full repeated	32.58	21.98	31.91	0.736	0.82662
FlexAttention	full repeated	28.58	18.01	27.91	0.746	0.82663
FlexAttention	low-res	20.58	13.91	19.91	0.827	0.82663

At 1 Mb, the low-res-bias FlexAttention path is useful: it avoids expanding the attention bias to the full repeated tensor, reduces observed peak memory by about 12 GiB relative to eager attention, and improves throughput by about 12%, with unchanged test differential Pearson at displayed precision.

4 Discussion

The most robust practical configuration is the combination of bfloat16, precomputed standardized convolutions, Triton int8 weight-only Conv1d, the 128 bp no-intermediates encoder path, and Triton no-indices max-pooling. This configuration kept peak memory below 48 GiB for both tested

input lengths while preserving test differential Pearson relative to the Triton Conv1d baseline. The fused high-resolution blocks are useful when minimizing observed memory is more important than maximizing throughput.

Linear-only quantization. The `torchao_float8_linear` and `bnb_nf4_weight_only_linear` strategies are implemented in `scripts/run_quant_ablation.py` and were benchmarked, but they are not included in the recommended stack. Linear-only float8 did not reduce actual device memory relative to `bf16_params` in the physical-batch runs, and was slower at batch 20: 11.50 examples/s versus 12.55 examples/s for `bf16_params`, both at about 36.2 GiB observed memory. NF4/NVFP4 linear and 1x1-convolution quantization helped in smaller weight-focused sweeps, but once wide convolutions are handled by the Triton int8 Conv1d path, the peak is dominated by activations, high-resolution convolution temporaries, pooling workspace, and allocator reservation. In the matched batch-32 combo run, adding NF4 linears and 1x1 convolutions to the Triton-conv stack used essentially the same observed memory as the bfloat16 Triton-conv stack, 43.62 GiB versus 43.61 GiB, with about a 0.001 drop in test differential Pearson.

FlexAttention at 131 kb. A parity-preserving FlexAttention replacement for `MHABlock` was implemented as `flex_mha`. It preserves the AlphaGenome score transform: additive attention bias, then tanh soft-cap, then softmax. Standalone parity was close, and full-model test differential Pearson was unchanged at displayed precision. At 131 kb it did not reduce peak memory, because the measured peak is dominated elsewhere after Triton convs are enabled. At batch 32, plain FlexAttention was slower than eager attention in the recorded full run, 8.44 versus 9.10 examples/s, with the same torch allocation. This makes FlexAttention context-dependent rather than a default 131 kb recommendation; the 1 Mb low-res-bias variant is the useful case.